

Lezione 13 - Python e C

CHIAMARE C DA PYTHON

◆ Idea

👉 E' possibile chiamare funzioni scritte in C direttamente da Python

👉 questo permette di:

- usare codice C *veloce*
- mantenere la semplicità di Python

✳️ combinazione di prestazioni e semplicità

◆ Problemi principali

👉 quando colleghiamo Python e C dobbiamo gestire:

- tipo degli argomenti
- tipo di ritorno
- gestione della memoria

✳️ questi aspetti NON sono automatici

PROBLEMI DI TIPO

◆ Differenza fondamentale

👉 i tipi in Python e C NON coincidono

◆ Interi

👉 in Python:

- tipo `int`
- precisione *arbitraria*

👉 in C:

- tipo `int`
- numero di bit fisso (32 o 64)

✳️ rappresentazioni diverse

◆ Caratteri

👉 in C:

- `char` rappresenta caratteri ASCII
- occupa 1 byte

👉 in Python:

- i caratteri sono Unicode
- possono occupare più byte

✳️ incompatibilità diretta

◆ Conseguenza

👉 quando passiamo dati tra Python e C:

- serve conversione dei tipi

✳️ altrimenti errori o comportamenti imprevisti

◆ Idea importante

👉 Python e C:

- usano modelli diversi per i dati

✳ serve un “ponte” tra i due mondi

📌 LIBRERIA CTYPES

◆ Idea

👉 Python fornisce la libreria **ctypes**

👉 permette di:

- accedere a librerie scritte in C
- convertire automaticamente i tipi
- chiamare funzioni C da Python

✳ è il *ponte* tra Python e C

◆ Caratteristiche

👉 **ctypes**:

- è inclusa nella libreria standard
- non richiede installazioni aggiuntive

👉 permette di:

- specificare il tipo degli argomenti
- specificare il tipo di ritorno
- effettuare conversioni automatiche

CONVERSIONE DEI TIPI

◆ Problema

👉 i tipi Python e C sono diversi

👉 quindi serve una corrispondenza tra:

- tipo C
- tipo ctypes
- tipo Python

◆ Tabella di conversione

👉 alcune corrispondenze importanti:

Tipo C	Tipo ctypes	Tipo Python
int	c_int	int
unsigned int	c_uint	int
float	c_float	float
double	c_double	float
char	c_char	<i>bytes (1 carattere)</i>
size_t	c_size_t	int

👉 **Attenzione:** `char` in C non corrisponde a una stringa Python ma a un oggetto *bytes*

◆ Significato

👉 quando chiamiamo una funzione C:

- Python usa i tipi **ctypes**
- effettua conversioni automatiche

✳ se i tipi sono compatibili

◆ Idea importante

👉 ctypes:

- risolve il problema dei tipi
- ma richiede di specificarli correttamente

✳ altrimenti errori difficili da individuare

📌 LIBRERIE CONDIVISE

◆ Idea

👉 per usare codice C in Python:

- dobbiamo compilare il codice C come **libreria condivisa**

👉 questo permette a Python di:

- caricare il codice
- chiamare le funzioni

◆ Compilazione

👉 per creare una libreria condivisa usiamo:



Codice:

```
gcc -fPIC -shared -o library.so library.c
```

◆ Significato

👉 opzioni:

- -fPIC → codice *position-independent*
- -shared → crea una libreria condivisa

👉 risultato:

- file `library.so`

◆ Caricamento in Python

👉 usiamo **ctypes**:



Codice:

```
ctypes.cdll.LoadLibrary("library.so")
```

◆ Risultato

👉 il valore restituito:

- è un oggetto Python
- permette di accedere alle funzioni C

◆ Idea importante

👉 il flusso è:

1. scrivere codice in C
2. compilare in libreria condivisa
3. caricare in Python
4. chiamare le funzioni

✨ integrazione tra Python e C (Flusso completo)

CHIAMARE FUNZIONI C DA PYTHON

◆ Idea

👉 una volta caricata la libreria:

- possiamo chiamare direttamente le funzioni C

👉 esempio:

Codice:

```
code.sum(3, 4)
```

👉 funziona se:

- i tipi sono compatibili

◆ Problema

👉 se i tipi non sono corretti:

- la conversione può fallire
- oppure dare risultati errati

✳ serve specificare i tipi

ARGOMENTI E VALORE DI RITORNO

◆ argtypes

👉 ogni funzione ha un attributo:

```
argtypes
```

👉 serve per specificare:

- il tipo degli argomenti

◆ Esempio



Codice:

```
code.multiply.argtypes = [ctypes.c_int, ctypes.c_float]
```

👉 significa:

- primo argomento → int
- secondo → float

◆ Comportamento

👉 se i tipi Python sono compatibili:

- la conversione avviene *automaticamente*

◆ restype

👉 per specificare il tipo di ritorno:

```
restype
```

◆ Esempio



Codice:

```
code.multiply.restype = ctypes.c_float
```

👉 significa:

- il risultato è un `float`

◆ Idea importante

👉 con **ctypes**:

- dobbiamo specificare i tipi
- Python gestisce la conversione

✳️ ma solo se i tipi sono corretti (altrimenti comportamento indefinito)

◆ Conseguenza

👉 se non specifichiamo i tipi:

- comportamento non garantito
- possibili errori difficili

✳️ sempre usare `argtypes` e `restype`

📌 PUNTATORI CON CTYPES

◆ Idea

👉 in C esistono i **puntatori**

👉 anche con **ctypes** possiamo:

- creare puntatori
- passarli alle funzioni

◆ Creare un valore



Codice:

```
a = ctypes.c_int(4)
```

👉 creiamo:

- una variabile C (`c_int`) con valore 4

◆ Ottenere un puntatore



Codice:

```
pa = pointer(a)
```

👉 otteniamo:

- un *puntatore* alla variabile `a`

◆ Tipo del puntatore

👉 per specificare il tipo:

```
POINTER(c_int)
```

👉 questo rappresenta:

- un puntatore a `int`

◆ Utilizzo

👉 possiamo:

- passare il puntatore a una funzione C
- lavorare direttamente su memoria

◆ Idea importante

👉 con **ctypes**:

- possiamo gestire i puntatori come in C

✳️ ma serve attenzione alla memoria

◆ Conclusione

👉 per usare C da Python servono:

- conversione dei tipi
- gestione dei puntatori
- uso corretto di **ctypes**

✳️ integrazione completa tra Python e C